

From SBOM to Action

Formerly known as: Beyond the Scan: Professional SBOM Management and Risk Mitigation with Dependency-Track and VEX

DevOpsDays Zurich / Marc Herren · 2026



Arriving early? Install the tools



Bring up the stack

Start it now — the vuln DB takes a few minutes

1. Follow the install.md instructions
2. Start Dependency Track with `docker compose up -d`
3. Leave it running while we talk



Who am I?

Who has an SBOM?

Show of hands

- ▶ Generated one this year?
- ▶ Did anything happen with it?

Easy to make. Hard to use.

- ▶ Every scanner spits one out
- ▶ Almost no one acts on them

Upcoming Cyber Resilience Act

- ▶ CRA mandates effective management
- ▶ Manufacturer obligations require a fast and pragmatic solution to report issues within 24h
 - Which applications, libraries or dependencies are affected
 - Risk management of known CVEs

Manufacturer Reporting Obligations

Timeline requirements for vulnerability and incident notifications

Actively Exploited Vulnerability

< 24 hours

Early Warning

Initial notification to CSIRT & ENISA

< 72 hours

Vulnerability Notification

General info, exploit nature, corrective measures

≤ 14 days

Final Report

After corrective/mitigating measure is available

Severe Security Incident

< 24 hours

Early Warning

Initial notification to CSIRT & ENISA

< 72 hours

Incident Notification

Nature, initial assessment, corrective measures

Within 1 month

Final Report

Within 1 month after incident notification

All notifications submitted via single reporting platform to designated CSIRT coordinator and ENISA

Today's menu

1. Introduction to Dependency Track
2. First steps
3. What is an SBOM
4. Your first SBOM
5. Scanning & risk awareness
6. Policy management
7. REST API
8. Exception handling / Risk assignment using VEX/VDR
9. Bonus: Ship it as OCI

1

Introduction to Dependency Track

Our toolstack of today

follow [Install.md](#) for instructions

Generate & manage

- ▶ **syft** to generate the SBOM
- ▶ **Dependency Track** is an intelligent Component Analysis platform that helps organizations identify and reduce risk in the software supply chain.

Scan & ship

- ▶ **grype** to scan an SBOM for vulnerabilities
- ▶ **oras** to handle OCI artifacts

Dependency Track local setup

Frontend
8080

API
8081

Postgres
5432

based on the local docker-compose setup

2

First steps

1. First login

localhost:8080

1. Open <http://localhost:8080>
2. Log in with the default credentials (admin/admin)
3. Take a quick tour: **Dashboard, Projects, Vulnerability Audit**
4. Navigate and check if the vulnerability sources have been updated successfully.
5. Which of the vulnerability sources are enabled by default? What is the update cycle? 6. Search for the latest **high** severity vulnerability and read the description.

1. First login — solution

localhost:8080

4. Navigate and check if the vulnerability sources have been updated successfully.
5. Which of the vulnerability sources are enabled by default? What is the update cycle?

SOLUTION

- ▶ Check Portfolio > Vulnerabilities. Details are in the log
- ▶ By default, only NIST is active as a vulnerability source
- ▶ Check Administration > Task scheduler > Components
 - By default the DBs are synchronized every 24h

3

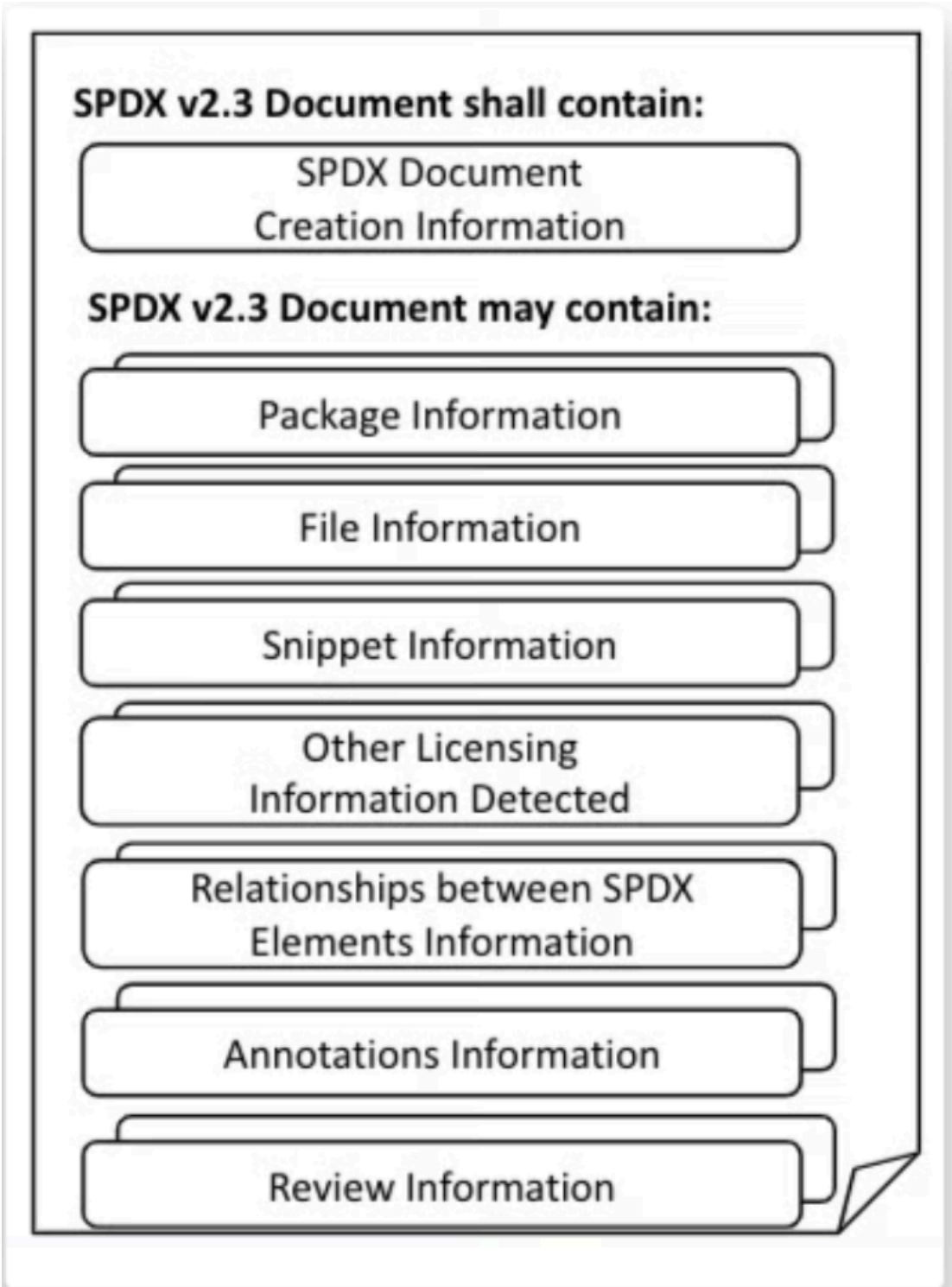
What is an SBOM

Two formats

Different goals, similar shape



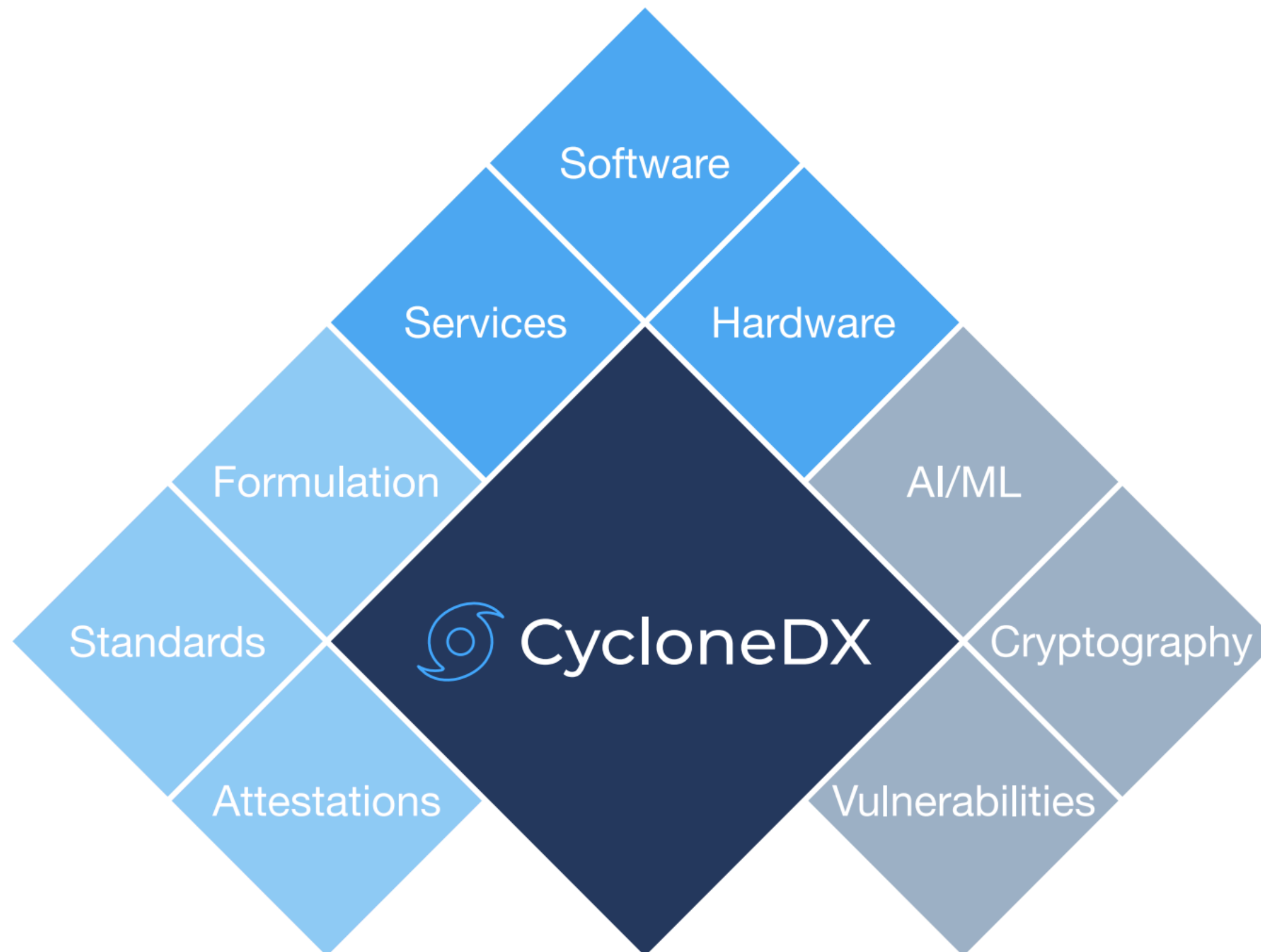
CycloneDX — security-first



SPDX — licensing-first

CycloneDX

<https://cyclonedx.org>



4

Your first SBOM

Generation ≠ scanning

- ▶ The bill of materials
- ▶ Lists each component in two formats: **CPE** and **PURL**
- ▶ Visualised with tools like Sunshine



Component	Depth	Depends on	Dependency of	Direct vulnerabilities	Transitive vulnerabilities	License
zlib	Search	Search Depen	Search Dependency of	Search Direct	Search Transi	Search Lic
zlib 1.2.13-r0	1, 2, 3, 4, 5	musl 1.2.3-r5	nginx 1.24.0-r1 libxml2 2.10.4-r0 curl 8.5.0-r0 libcurl 8.5.0-r0 freetype 2.12.1-r0 tiff 4.4.0-r4 libpng 1.6.39-r0 apk-tools 2.12.10-r1 nginx-module-njs 1.24.0.0.7.12-r1	-	-	Zlib

Sunshine output

CPE & PURL

How a component is identified

CPE (Common Platform Enumeration)

```
cpe:2.3:a:zlib:zlib:1.2.13-r0:*:*:*:*:*:*:*
```

PURL (Package URL)

```
pkg:apk/alpine/zlib@1.2.13-r0?arch=aarch64&distro=alpine-3.17.7
```

A PURL is a specially formatted URL that describes a software package's location or identity in a package registry.

grype

Generation ≠ scanning

- ▶ A vulnerability scanner for container images and filesystems.
- ▶ Judge components against a or multiple vulnerability databases
 - NVD (National Vulnerability Database)
 - GitHub Security Advisories (GHSA)
 - Alpine SecDB
 - Amazon Linux
 - ...



2. syft

1. Explore the component list:

```
syft nginx:1.24-alpine -o table
```

2. Generate the CycloneDX SBOM:

```
syft nginx:1.24-alpine -o cyclonedx-json=alpine.cdx.json
```

Optional: Analyze the file with Sunshine <https://cyclonedx.github.io/Sunshine/>

5

Scanning & risk awareness

3. gype

1. Scan the SBOM with gype:

```
gype alpine.cdx.json
```

2. Optional: Analyze the SBOM with other tools (trivy, cve-bin-tool,...) and compare the output

EPSS, severity, risk

<https://nvd.nist.gov/vuln-metrics/cvss/v4-calculator>

- ▶ **CVSS** — how bad it is
- ▶ **EPSS** — how likely it gets exploited
- ▶ **Risk** — Dependency Track's combined signal

CVSS v4.0 calculator

CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:N/SC:N/SI:N/SA:N

Reset

CVSS v4.0 Score: 0 / None ⓘ

Base Metrics ?

Exploitability Metrics

Attack Vector (AV): **Network (N)** **Adjacent (A)** **Local (L)** **Physical (P)**

Attack Complexity (AC): **Low (L)** **High (H)**

Attack Requirements (AT): **None (N)** **Present (P)**

Privileges Required (PR): **None (N)** **Low (L)** **High (H)**

User Interaction (UI): **None (N)** **Passive (P)** **Active (A)**

Vulnerable System Impact Metrics

Confidentiality (VC): **High (H)** **Low (L)** **None (N)**

Integrity (VI): **High (H)** **Low (L)** **None (N)**

Availability (VA): **High (H)** **Low (L)** **None (N)**

Subsequent System Impact Metrics

Confidentiality (SC): **High (H)** **Low (L)** **None (N)**

CVSS Scoring → Severity

Scores range from 0 – 10

None	0.0
Low	0.1 – 3.9
Medium	4.0 – 6.9
High	7.0 – 8.9
Critical	9.0 – 10.0

grype output

Severity, EPSS percentile, and risk score for each finding

```
grype alpine.cdx.json
✓ Vulnerability DB [no update available]
✓ Scanned for vulnerabilities [160 vulnerability matches]
  └─ by severity: 11 critical, 56 high, 80 medium, 13 low, 0 negligible
  └─ by status: 66 fixed, 94 not-fixed, 0 ignored
```

NAME	INSTALLED	FIXED IN	TYPE	VULNERABILITY	SEVERITY	EPSS	RISK
freetype	2.12.1-r0		apk	CVE-2025-27363	High	69.2% (98th)	81.9
nghttp2-libs	1.51.0-r2		apk	CVE-2024-28182	Medium	25.0% (96th)	12.9
libcrypto3	3.0.12-r4	3.0.15-r0	apk	CVE-2024-6119	High	6.6% (91st)	4.9
libssl3	3.0.12-r4	3.0.15-r0	apk	CVE-2024-6119	High	6.6% (91st)	4.9
libcrypto3	3.0.12-r4	3.0.14-r0	apk	CVE-2024-5535	Critical	5.1% (89th)	4.6
libssl3	3.0.12-r4	3.0.14-r0	apk	CVE-2024-5535	Critical	5.1% (89th)	4.6
libcrypto3	3.0.12-r4	3.0.12-r5	apk	CVE-2024-2511	Medium	4.1% (88th)	2.3
libssl3	3.0.12-r4	3.0.12-r5	apk	CVE-2024-2511	Medium	4.1% (88th)	2.3

Scanner mess

Everyone shows you something different



<https://www.youtube.com/watch?v=lbPX-WYTjNc>

Add the SBOM to Dependency Track

- ▶ In order to add an SBOM to Dependency Track you need a Project first
- ▶ A Project can be a Parent/Child structure where findings are aggregated

+ Create Project ☒ Show inactive projects ☐ Show flat project view Search ↺ ☰										
Project Name	Tags	Version	Latest	Classifier	Last BOM Import	BOM Format	Risk Score	Active	Policy Violations	Vulnerabilities
▼ Awesome Product	Aggregate all direct children			Application	-	-	62	☒	65	10 4
▼ Awesome Product - Web UI	Aggregate by Tag "example"			Application	-	-	0	☒	0	0
Awesome Product - Web UI		qa		Application	4 Oct 2024 at 12:05:29	CycloneDX 1.5	21	☒	0	3 2
Awesome Product - Web UI		prod		Application	4 Oct 2024 at 12:04:43	CycloneDX 1.5	16	☒	0	2 2
Awesome Product - Web UI	example	dev	☒	Application	4 Oct 2024 at 12:04:57	CycloneDX 1.4	0	☒	0	0
▼ Awesome Product - Backend	Aggregate latest version of direct children			Application	-	-	46	☒	65	8 2
▶ Awesome Product - Backend - QA	Aggregate all direct children			Application	-	-	21	☒	0	3 2
▶ Awesome Product - Backend - prod	Aggregate all direct children			Application	-	-	20	☒	0	4
▼ Awesome Product - Backend - Dev	Aggregate all direct children		☒	Application	-	-	46	☒	65	8 2
Awesome Product - Backend - MicroService B		dev	☒	Application	4 Oct 2024 at 12:36:41	CycloneDX 1.5	20	☒	54	4
Awesome Product - Backend - MicroService A		dev	☒	Application	4 Oct 2024 at 12:37:19	CycloneDX 1.5	26	☒	11	4 2
▼ Awesome Product - Android App	Aggregate latest version of direct children			Application	-	-	16	☒	0	2 2
Awesome Product - Android App		1.0.3	☒	Application	4 Oct 2024 at 12:05:59	CycloneDX 1.5	16	☒	0	2 2
Awesome Product - Android App		1.0.1		Application	4 Oct 2024 at 12:02:55	CycloneDX 1.5	52	☒	0	8 4
Awesome Product - Android App		1.0.0		Application	4 Oct 2024 at 12:03:12	CycloneDX 1.5	11	☒	0	1 2

Your first project

1. Create a Project
2. Manually add the SBOM to the project
3. Search for any CVE findings

SOLUTION

Pay attention that in a parent/child project, the Parent text box is a search field!

6

Policy management

Policy Compliance

There are three types of policy violations:

- ▶ License
- ▶ Security
- ▶ Operational

Operational Violation:

- ▶ Age (publish date)
- ▶ Coordinates (group, name, version)
- ▶ Package URL
- ▶ CPE
- ▶ SWID Tag ID
- ▶ Hash
- ▶ Version distance

4. Your first policy

Administration → Policy Management

1. **Create Policy**, pick condition:
 - ▶ CVE severity $\geq$ High, or
 - ▶ License = GPL
2. Set the violation state to **Fail**
3. Check the project for violations

SOLUTION

The project is only updated once a new SBOM is submitted, not on existing ones

7

REST API

REST API

<https://docs.dependencytrack.org/usage/cicd/>

- ▶ A full REST API to handle automated CI ingestion of SBOMs

Explore the REST API

localhost:8081

1. Access the API http port
2. Analyze the /v1/bom schema
3. Extract the schema options for this API
 - ▶ You can use a visualizer of choice or <https://editor.swagger.io/>

5. POST /api/v1/bom

The same call your CI makes

1. Create a new project in the UI and copy its ProjectID (Under details, Object Identifier)
2. **Administration** → **Access Management** → **Teams** → **Automation** → generate API key
3. Upload via curl:

```
curl -X POST http://localhost:8081/api/v1/bom \  
  -H "X-API-Key: <your-api-key>" \  
  -F "project=<your-project-uuid>" \  
  -F "bom=@alpine.cdx.json"
```

4. Analyze the project as well as your policy
5. Explore how to enable the autoCreate capability (Permissions, Curl command)

8

Exception handling / Risk
assignment using VEX/VDR

Two formats, similar goal

<https://cyclonedx.org/capabilities/vex/>

VDR (Vulnerability Disclosure Report)

- ▶ **What's vulnerable**
- ▶ Disclosure-oriented list
- ▶ Shipped as a transparency artifact

VEX (Vulnerability Exploitability eXchange)

- ▶ **What it means**
- ▶ `affected` / `not_affected` / `under_investigation`
- ▶ Technically a subset of VDR
 - A VDR might list 10 CVEs; the accompanying VEX might then say 8 are `not_affected` and 2 are `affected`.

VEX formats

why one if you can have many?

OpenVEX is an implementation of VEX that is designed to be minimal, compliant, interoperable, and embeddable. <https://github.com/openvex>

Dependency Track supports only the CycloneDX VEX format, while local scanners like `grype` only support OpenVEX

Add a CISO

1. **Administration → Access Management → Teams**
2. Create a new Team named `CISO`
3. Add the rights for `VULNERABILITY` management
4. Create a new user and add it to this team

6. Risk management

1. **Project > Audit Vulnerabilities** — pick at least one critical CVE
2. **Analysis** → set to **Not Affected** with a comment
3. **Export VEX** → save as `vex.cdx.json`
4. Analyze the VEX document

SOLUTION

```
"analysis" : {  
  "state" : "not_affected",  
  "justification" : "protected_at_runtime"
```

9

Bonus: Ship it as OCI

Bundle it with the artifact

<https://github.com/oras-project/oras>

- ▶ OCI artifact = blob + manifest
- ▶ SBOM, VEX/VDR, signatures, attestations
- ▶ Attestation vs. Referrer

	Attestation	Referrer
VEX update requires	Re-sign the image	Attach a new referrer
Signed image changes	✅ Yes	❌ No
Metadata lifecycle	Tied to image release	Independent

7. Bundle and push

Look first

1. See what attestations are already attached to the public image:

```
oras discover --platform linux/amd64 \  
  docker.io/library/nginx:1.24-alpine
```


7. Bundle and push

Bundle the artifacts

2. Pull the image into a local OCI layout:

```
oras copy --platform linux/amd64 \  
  docker.io/library/nginx:1.24-alpine \  
  --to-oci-layout ./nginx-layout:nginx-amd64
```

3. Attach the SBOM:

```
oras attach --oci-layout \  
  --artifact-type application/vnd.cyclonedx+json \  
  ./nginx-layout:nginx-amd64 \  
  alpine.cdx.json:application/vnd.cyclonedx+json
```

7. Bundle and push

Verify locally

4. Attach the VEX:

```
oras attach --oci-layout \  
  --artifact-type application/vnd.cyclonedx.vdr+json \  
  ./nginx-layout:nginx-amd64 \  
  vex.cdx.json:application/vnd.cyclonedx+json
```

5. Inspect the referrer graph:

```
oras discover --oci-layout ./nginx-layout:nginx-amd64
```

Thank you

marc@remmen.io

 Stay in touch

